

BCC1313 ALGORITHMS AND COMPLEXITY

Stack Operations

INTRODUCTION

- A **stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle.
- If you insert a series of data into a stack and then removed it, the order of the data will be reversed.
- We encounter various types of stacks in our daily lives, such as a stack of coins or a stack of dishes.





STACKS

- A **stack** stores data similarly to an **array**, with the following three key constraints:
    1. Data can only be **inserted at the end of a stack.**
    2. Data can only be **read from the end of a stack**.
    3. Data can only be **removed from the end of a stack.**

STACKS

- A stack is a special case of a list in which insertion and deletion are made only at the first position of the list, called **top.**

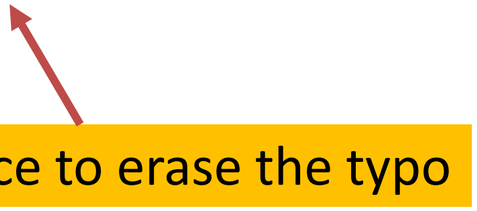
STACKS APPLICATIONS

Example 1:

- A program to read a line typed from a keyboard.

WELCOOME

WELCOO[←]ME



STACKS APPLICATIONS

- Initial draft of a solution of this program:

```
while(not end of line)
{  Read a new character ch
    if(ch is not a '←')
       add ch to the ADT
    else if (the ADT is not empty)
       remove from the ADT the item
          that was added most recently
    else
       Ignore the '←'
}
```

STACKS OPERATIONS

- ADT stack operations:
  - **Create an empty stack**
  - **Destroy a stack**
  - **Determine whether a stack is empty**
  - **Add a new item to the stack**
  - **Remove the item that was added most recently**
  - **Retrieve the item that was added most recently.**

STACKS OPERATIONS

- Operation contract for the ADT stack:

```
// Create an empty stack
void createStack();

// Destroy the stack
void destroyStack();

// Determine whether a stack is empty
bool isEmpty() const;

// Add a new item to the stack
void push(int newItem);

// Remove the item that was added most recently
void pop();

// Retrieve the item that was added most recently
int getTop() const;
```

STACKS OPERATIONS

- Use the ADT stack's operations for the previous problem:

```
+readAndCorrect(out aStack:Stack)

  aStack.createStack()
  Read newChar
  while(newChar is not the end-of-line symbol)
  {  if(newChar is not '←')
       aStack.push(newChar)
     else if(!aStack.isEmpty())
       aStack.pop()

     Read newChar
  }
```

STACKS APPLICATIONS

Example 2:

- Checking for balanced braces.
  - In most programming languages, braces { and } are used to delimit groups of statements (blocks).

```
if (conditions)
    { true statements; }
else
    { false statements;}
```

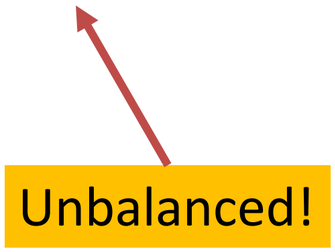
STACKS APPLICATIONS

- An example of balanced braces:

```
abc{defg{ijk}{l{mn}}op}qr
```

- An example of unbalanced braces:

```
abc{def}}{ijkl{mn}op}qr
```



STACKS APPLICATIONS

- Requirements for balanced braces:
  - Traverse the string from left to right.
    - Each time a } is encounter, it must match a previously encountered {
    - When the end of the string is reach, every { must have a matching }

STACKS APPLICATIONS

- Initial draft of a solution of this program:

```
aStack.createStack()
balancedSoFar = true
i=0

while (balancedSoFar and i < length of aString)
{   ch = character at position i in aString

    if (ch is '{')
        aStack.push('{')
```

STACKS APPLICATIONS

- Initial draft of a solution of this program:

```
    else if (ch is '}')
        if(!aStack.isEmpty())
            aStack.Pop()
        else
            balancedSoFar = false
} // end while

if (balancedSoFar and astack.isEmpty())
    aString has balanced braces
else
    aString does not have balanced braces
```

Input string	Stack as algorithm executes			
	1.	2.	3.	4.
{a{b}c}	{	{ {	{	
{a{bc}	{	{ {	{	
{ab}c}	{			

1. Push {
2. Push {
3. Pop
4. Pop

Stack empty → balanced

1. Push {
2. Push {
3. Pop

Stack not empty → unbalanced

1. Push {
2. Pop

Stack empty when last } encountered → unbalanced

STACKS APPLICATIONS

Example 3:

- Recognizing strings in a language
  - $L = \{w\$w : w$ *is a possibly empty string of characters other than* \$, $w' = reverse\ (w)\}$

A$A

ABC$CBA

AB$AB

The second half is not the reverse of the first half

STACKS APPLICATIONS

- A solution using a stack:
  - Traverse the first half of the string, **pushing** each character onto a stack.
  - Once reach the $, for each character in the second half of the string, match a **popped** character of the stack.

STACKS APPLICATIONS

- The algorithm of L reorganization:

```
aStack.createStack()

i=0
ch = character at position i in aString

while(ch is not '$')
{   aStack.push(ch)
    ++i
    ch = character at position i in aString
}

// skip the $
++i
```

STACKS APPLICATIONS

```
inLanguage = true    // assume the string is in
                     // language
While(inLanguage and i < length of aString)
{   aStack.pop(stackTop)
    ch = character at position i in aString
    if(stackTop equals ch)
        ++i
    else
        inLanguage = f
}

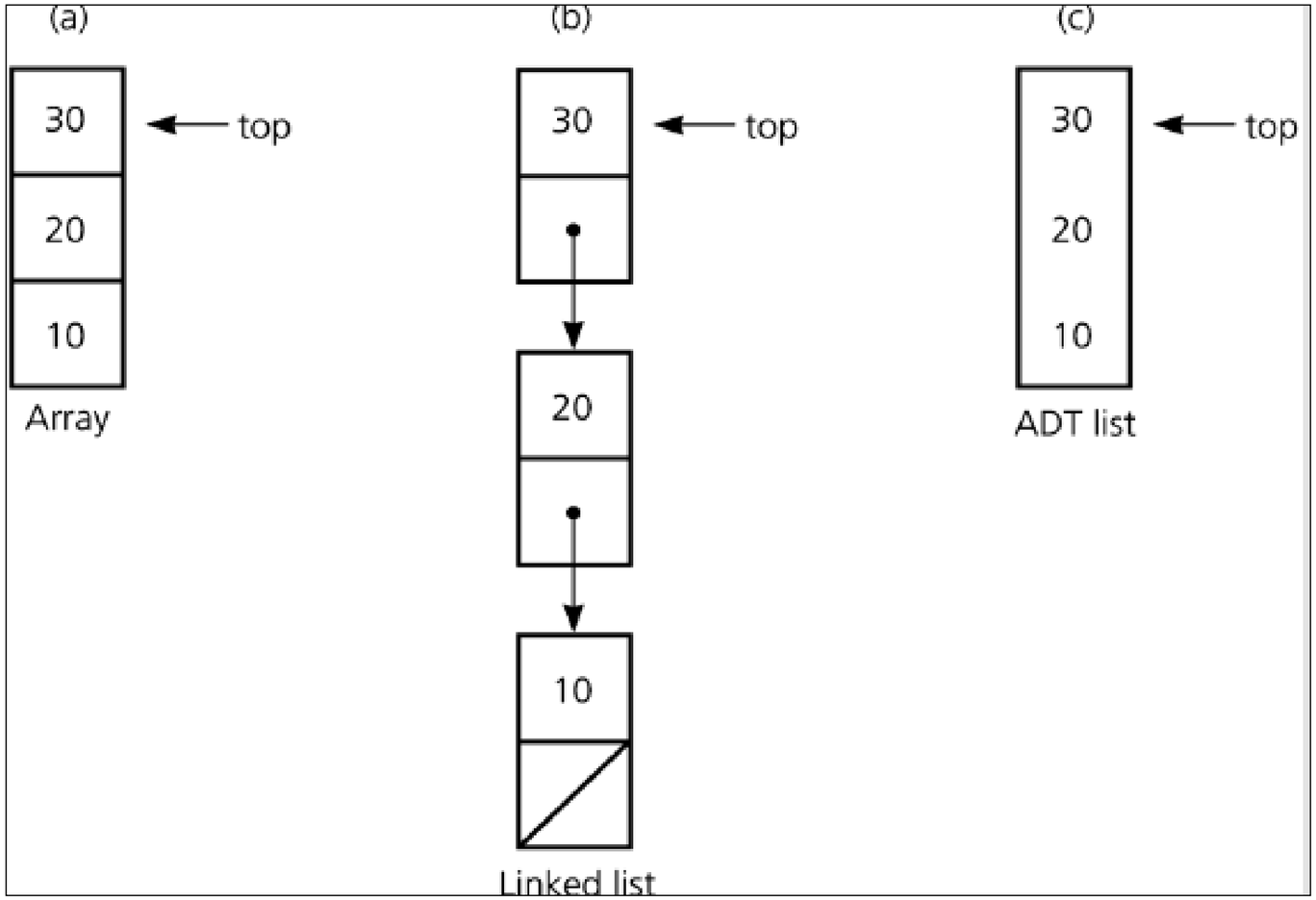
if(inLanguage and aStack.isEmpty())
    aString is in Language
else
    aString is not in Language
```

IMPLEMENTATION

- The ADT stack can be implemented using:
  - An array
  - A linked list
  - The ADT list

IMPLEMENTATION

- Example:



IMPLEMENTATION

- **Comparing implementations**

1. **Fixed versus dynamic size**:

    - A **statically** allocated array-based implementation.
        - The size is known in advance, e.g. 80 characters in one line.
        - Prevent the push operation from adding an item to the stack, if the array is full.
    - A **dynamically** allocated array-based implementation or a pointer-based implementation.
        - No size restriction on the stack.
        - Copy data between dynamic arrays.

IMPLEMENTATION

- **Comparing implementations**
  2. **A pointer-based implementation vs one that uses a pointer-based implementation of the ADT list:**
     - Pointer-based implementation is more efficient
     - ADT list approach reuses an already implemented class.
       - Much simpler to write
       - Saves programming time

STACKS APPLICATIONS

Example 4:

- Algebraic Expression
  - Stacks are widely used in the conversion and evaluation of algebraic expressions. The different ways of writing algebraic expressions include:
    - **infix expression**
    - **prefix expression**
    - **postfix expression**

STACKS APPLICATIONS

1. **Infix Expression:**
    - Operators are placed between operands (e.g., A + B).
2. **Prefix Expression:**
    - Operators are placed before their operands (e.g., + A B).
3. **Postfix Expression:**
    - Operators are placed after their operands (e.g., A B +).

STACKS APPLICATIONS

1. Infix Expression

- Ambiguity of infix expressions:

  a + b * c

  - **a + b then * c?**
  - **b * c then + a?**

- Infix expression require **precedence rules**, **associativity rules**, and the use of **parentheses** to avoid ambiguity

STACKS APPLICATIONS

- **Precedence rules** determine which operators should be evaluated earlier.

  *** precede +, so a + b * c → a + (b*c)**

- **Equal precedence:**

  a / b * c

  - **(a / b) * c?**
  - **a / (b * c)?**

STACKS APPLICATIONS

- **Associativity rules** specify the evaluation sequence of operators with the same precedence.
  - For binary operator, generally ‘left-to-right’ association.

    a / b * c = (a / b) * c

  - If need other interpretation, use parenthesis:

    (a + b) / c

STACKS APPLICATIONS

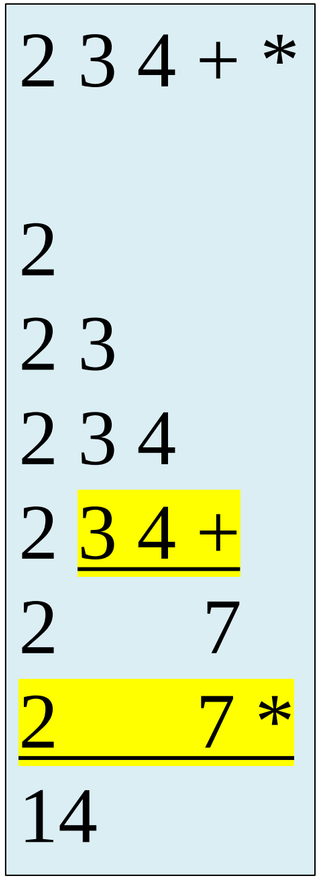
- The advantage of prefix and postfix expression is that they never need precedence rules, association rules, and parenthesis.
  - They can avoid the ambiguity of the evaluation of infix expressions.
  - Example:

a b + c * = (a + b) c *

2 3 4 + * = 2 (3 4 +) *

STACKS APPLICATIONS

- **Evaluating Postfix Expressions**
  - Only consider the binary operators *, /, +, and - (left-to-right association).
  - A postfix calculator:
    - An operator is always applied to the two operands that immediately precede it.



When encounter an operator, the calculator must be able to retrieve the operands entered most recently.

STACKS APPLICATIONS

- **Evaluating postfix using stack:**
  - When an operand is entered, the calculator
    - ➢ pushes it onto a stack.
  - Ehen an operator is entered, the calculator
    - ➢ pops the top two operands from the stack
    - ➢ Evaluate the operation
    - ➢ Pushes the result of the operation onto the stack

STACKS APPLICATIONS

- A pseudocode algorithm that evaluates postfix expressions:

```
for (each character ch in the string)
{ if (ch is an operand)
        push ch onto stack
  else
  {     // ch is an operator
        operand2 = top of stack
        pop the stack

        operand1 = top of stack
        pop the stack

        result = operand1 ch operand2
        push result onto stack
  }
}
```

STACKS APPLICATIONS

- Calculate **2 3 4 + ***

Key entered	Calculator action		After stack operation: Stack (bottom to top)
2	push 2		2
3	push 3		2 3
4	push 4		2 3 4
+	operand2 = pop stack	(4)	2 3
	operand1 = pop stack	(3)	2
	result = operand1 + operand2	(7)	2
	push result		2 7
*	operand2 = pop stack	(7)	2
	operand1 = pop stack	(2)	
	result = operand1 * operand2	(14)	
	push result		14

STACKS APPLICATIONS

- **Converting infix expression to equivalent postfix expression**
  - By creating a programs, a compilers will translate infix expressions into postfix form for computer execution.
  - Process - Compilers convert infix expressions into equivalent postfix expressions.
  - Evaluation - Computers can evaluate the postfix expression using the previously established algorithm.

STACKS APPLICATIONS

- **Converting infix expression to equivalent postfix expression**
  - The infix expression here allow:
    - Parentheses
    - Operator precedence
    - Left-to-right association
  - Example of infix expression:

a – (b + c * d) / e

STACKS APPLICATIONS

- Fact about converting from infix to postfix:

  2 * (3 + 4) → 2 3 4 + *

  - Operands always stay in the same order with respect to one another.
  - An operator will move only to the right with respect to the operands
  - All parentheses are removed.

STACKS APPLICATIONS

- Fact Rules for Postfix Conversion Using Stack:
  - Operands: Directly added to the postfix expression.
  - Operators:
    - Pushed onto the stack.
    - If an operator has higher or equal precedence than the operator on top of the stack, pop the operator from the stack and add it to the postfix expression before pushing the new operator.

STACKS APPLICATIONS

- Factors that determines the placement of the operators in the postfix expression:
  - Parentheses
    (a + b) * c → a b + c *
    a + b * c → a b c * +
  - Precedence
    a * b + c → a b * c +
    a + b * c → a b c * +
  - Left-to-right association
    a + b - c → a b + c -
    a - b + c → a b - c +

STACKS APPLICATIONS

- Example of converting the infix expression: **a – (b + c * d) / e**

ch	Stack (bottom to top)	postfixExp	
a		a	
–	–	a	
(	– (	a	
b	– (	ab	
+	– (+	ab	
c	– (+	abc	
*	– (+ *	abc	
d	– (+ *	abcd	
)	– (+	abcd*	Move operators from stack to **postfixExp** until "("
	– (	abcd*+	
	–	abcd*+	
/	–/	abcd*+	
e	–/	abcd*+e	Copy operators from stack to **postfixExp**
		abcd*+e/–	

STACKS APPLICATIONS

- The algorithm:

```
for (each character ch in the infix expression)
{  switch (ch)
   {
      case operand: // append operand to end of PE
         postfixExp = postfixExp + ch
         break

      case '(':     // save '(' on stack
         aStack.push(ch)
         break

      case ')': // pop stack until matching '('
         while (top of stack is not '(')
         {  postfixExp = postfix + (top of aStack)
            aStack.pop()
         }
         aStack.pop()   // remove the ')'
         break
```

STACKS APPLICATIONS

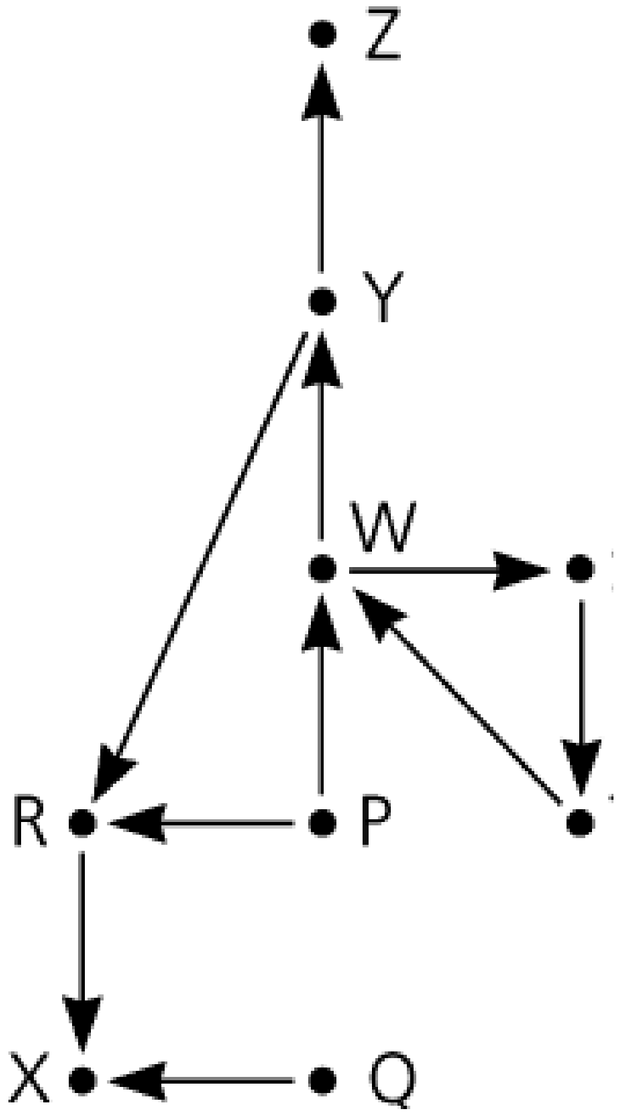
```
        case operator:

            while (!aStack.isEmpty() and
                   top of stack is not '(' and
                   precedence(ch) <= precedence(top of aStack))
                  { postfixExp = postfixExp + (top of aStack)
                    aStack.pop()
                  }
            aStack.push(ch)   // save new operator
            break
    } // end switch
} // end for
// the end of the infix expression
// append to postfixExp the operators remaining in the stack
while (!aStack.isEmpty())
{  postfixExp = postfixExp + (top of aStack)
   aStack.pop()
} // end while
```

STACKS APPLICATIONS

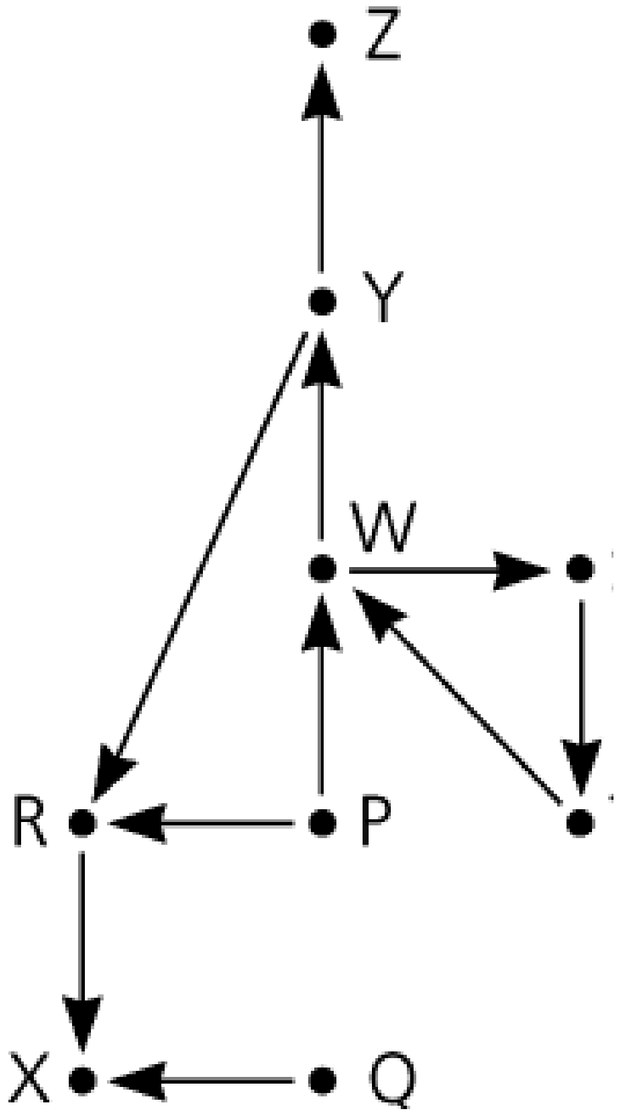
Example 5:

- A Search Problem
  - Backtracking can be used to recover from choosing a wrong search.



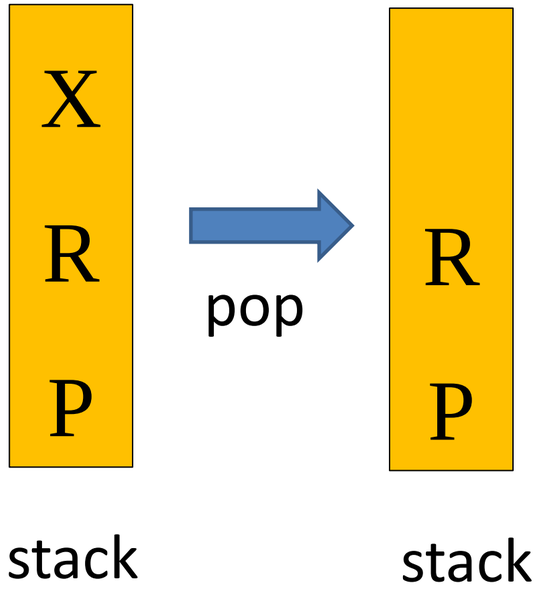
STACKS APPLICATIONS

- Example of backtracking:
  - Origin: P
  - Destination: Z
  - Current search path: P → R → X
    - It is a mistake to go to X (problem of circling)
    - Backtrack to R and try other possibilities



STACKS APPLICATIONS

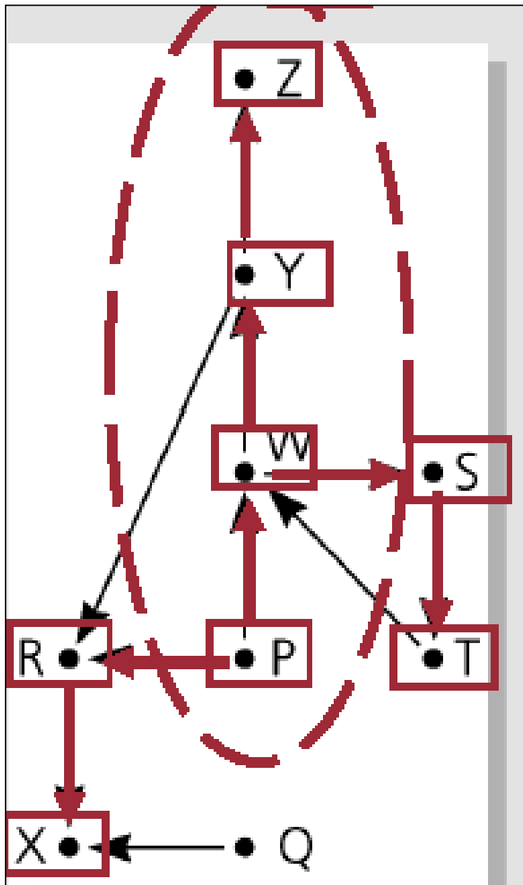
- How to backtrack?
  - Each time visit, push onto the stack
  - Backtrack: pop from the stack



STACKS APPLICATIONS

- To avoid circling:
  - Mark the visited point
  - When choosing the next point, restrict to unmarked points adjacent to the current point

STACKS APPLICATIONS



Action	Reason	Contents of stack (bottom to top)
Push P	Initialize	P
Push R	Next unvisited adjacent city	P R
Push X	Next unvisited adjacent city	P R X
Pop X	No unvisited adjacent city	P R
Pop R	No unvisited adjacent city	P
Push W	Next unvisited adjacent city	P W
Push S	Next unvisited adjacent city	P W S
Push T	Next unvisited adjacent city	P W S T
Pop T	No unvisited adjacent city	P W S
Pop S	No unvisited adjacent city	P W
Push Y	Next unvisited adjacent city	P W Y
Push Z	Next unvisited adjacent city	P W Y Z

SUMMARY

- A stack is a linear data structure that follows the Last In, First Out (LIFO) principle, meaning the last element added is the first to be removed.
- Example of applications:
  - **Expression Evaluation**: evaluating arithmetic expressions (infix, postfix, prefix).
  - **Expression Conversion**: converting infix expressions to postfix or prefix
  - **Backtracking Algorithms**: Utilized in algorithms like depth-first search and backtracking problems.
  - **Function Call Management**: Maintains function calls and local variables during execution in programming languages.